

Memoria Práctica 1

1. Introducción

En esta práctica hemos diseñado una API REST para que los usuarios puedan tener y compartir ficheros. Hemos aprendido más acerca de Python y sus utilidades, así como aprender a enviar y recibir peticiones HTTP. También hemos adquirido los conocimientos para desplegar nuestro código con docker separado según los microservicios desarrollados. Los microservicios `user.py` y `file.py` se levantan cada uno en su contenedor desde el mismo `docker-compose`. Toda la funcionalidad se comprueba con el cliente de pruebas en `cliente.py`.

2. Servicio de usuarios (`user.py`)

Expone `/user` con tres métodos en el puerto 5050: PUT se le pasa el nombre y contraseña del usuario y si consigue crear el usuario devuelve el código 201 así como el uid y token. En cambio en caso de que ya existe devuelve el código 409). POST inicia sesión, se le pasa nombre y contraseña del usuario y devuelve el código 200 si tiene éxito, o 401 si las credenciales no son correctas y por último, PATCH, al que se le pasa la contraseña y en caso de éxito cambia la contraseña y devuelve el código 200. En caso de error dependiendo del tipo devuelve 400 o 401.

La contraseña se guarda como hash SHA-256. El UID se genera con `uuid4()` y el token con `uuid5(secret_uuid, uid)`. `secret_uuid` se genera una sola vez y se guarda en `shared/secret_uuid.txt`, que también lee el servicio de ficheros.

3. Servicio de ficheros (`file.py`)

Cada usuario tiene su carpeta `file/<uid>/`, y cada documento es un fichero de texto dentro de ella. La visibilidad se guarda en `visibilidad/<uid>.json` como `{nombre: true/false}` y por defecto todo es privado. Para comprobar el token, se recalcula `uuid5(secret, uid de la ruta)` y se compara con el que llega en la cabecera. Las acciones de listar, crear, borrar y cambiar la visibilidad exige token, pero el GET de ficheros públicos no. Acerca de los métodos, comparten el sistema de códigos de error, devolviendo 401 si no hay autorización, 404 si el documento no existe o 400 si el cuerpo no es válido.

De nuestra implementación cabe destacar que al borrar un documento también se quita su entrada del fichero de visibilidad. Así, si luego se crea otro con el mismo nombre, vuelve a ser privado por defecto. El servicio de ficheros no consulta al de usuarios para validar el token. Le basta con conocer el `secret_uuid` compartido, y así los dos servicios siguen desacoplados, el único requisito es que `user.py` se haya desplegado antes de que `file.py` empiece a responder peticiones y necesite el token secreto que `user` genera.

4. Cliente de pruebas (`cliente.py`)

En `cliente.py` están las pruebas para comprobar que funciona toda la funcionalidad y que como se puede ver en las capturas, pasan todas las pruebas. Para hacer las peticiones usamos dos funciones, “`peticion`” (usuarios) y “`peticion_file`” (ficheros). Las dos añaden la cabecera `Authorization: Bearer` cuando se les pasa un token. La función “`comprobar`”

compara el código recibido con el esperado y, cuando hace falta, también el contenido de la respuesta: por ejemplo, que el login devuelva el mismo UID y token, o que el GET devuelva el texto escrito.

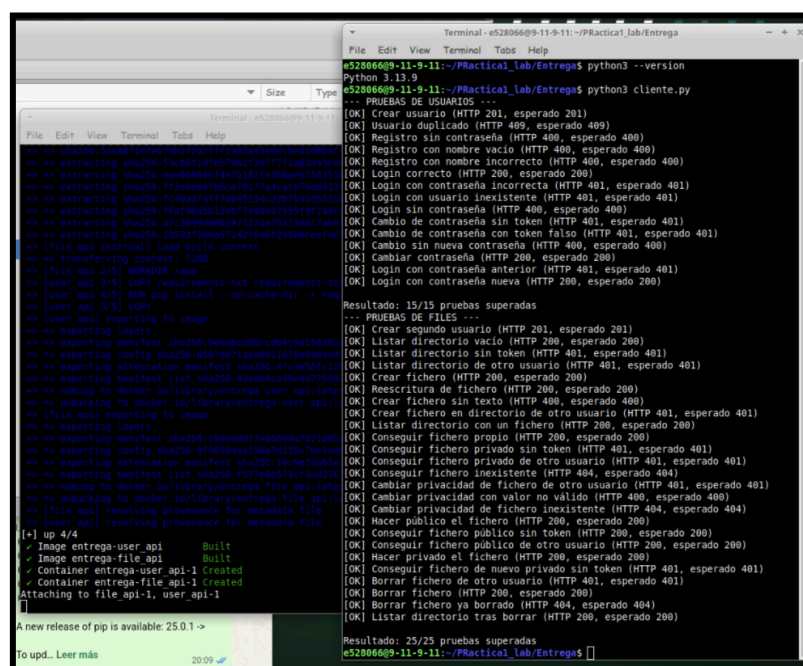
5. Persistencia

Acerca de los usuarios, hemos tomado la decisión de diseño de que éstos se guarden en `user_data/users.json`, un diccionario indexado por nombre con el UID y el hash de la contraseña de cada uno. Se eligió un fichero JSON porque nos pareció conveniente al estar usándolo para las peticiones HTTP. De los documentos, cada uno es un fichero de texto en `file/<uid>/<nombre>`, así que listar el directorio es simplemente listar esa carpeta. Sobre la visibilidad, se ha creado una carpeta aparte `visibilidad/<uid>.json`. Está separada de los documentos para que no aparezca al listar el directorio. Si un documento no tiene entrada en ese fichero, se considera privado. Por último, hay una carpeta llamada `shared/secret_uid.txt`. Lo genera `user.py` la primera vez que arranca y lo leen los dos servicios.

6. Docker

Hay una única imagen (`python:3.13.9`) que instala `requirements.txt` y expone los puertos 5050 y 5051. El `docker-compose.yml` levanta dos servicios con la misma imagen, cada uno con su comando (`user.py` / `file.py`) y su puerto. Así, cada microservicio es un contenedor independiente.

Los dos contenedores montan el directorio del proyecto como volumen. Así comparten el fichero `shared/secret_uid.txt` y los datos se conservan aunque se paren los contenedores. La opción `network: host` en la construcción la añadimos para funcionase también en los ordenadores del laboratorio.



```
Terminal - e5286609-11-9-11-~/Practical_lab/Entrega
File Edit View Terminal Tabs Help
e5286609-11-9-11-~/Practical_lab/Entrega$ python3 --version
Python 3.13.9
e5286609-11-9-11-~/Practical_lab/Entrega$ python3 cliente.py
... PRUEBAS DE USUARIOS ...
[OK] Crear usuario (HTTP 201, esperado 201)
[OK] Usuario duplicado (HTTP 409, esperado 409)
[OK] Registro sin contraseña (HTTP 400, esperado 400)
[OK] Registro con nombre vacío (HTTP 400, esperado 400)
[OK] Registro con nombre incorrecto (HTTP 400, esperado 400)
[OK] Login correcto (HTTP 200, esperado 200)
[OK] Login con contraseña incorrecta (HTTP 401, esperado 401)
[OK] Login con usuario inexistente (HTTP 401, esperado 401)
[OK] Login sin contraseña (HTTP 400, esperado 400)
[OK] Cambio de contraseña sin token (HTTP 401, esperado 401)
[OK] Cambio de contraseña con token falso (HTTP 401, esperado 401)
[OK] Cambio sin nueva contraseña (HTTP 400, esperado 400)
[OK] Cambio contraseña (HTTP 200, esperado 200)
[OK] Login con contraseña anterior (HTTP 401, esperado 401)
[OK] Login con contraseña nueva (HTTP 200, esperado 200)
Resultado: 15/15 pruebas superadas
... PRUEBAS DE FILES ...
[OK] Crear segundo usuario (HTTP 201, esperado 201)
[OK] Listar directorio vacío (HTTP 200, esperado 200)
[OK] Listar directorio sin token (HTTP 401, esperado 401)
[OK] Listar directorio de otro usuario (HTTP 401, esperado 401)
[OK] Crear fichero (HTTP 200, esperado 200)
[OK] Reescritura de fichero (HTTP 200, esperado 200)
[OK] Crear fichero sin texto (HTTP 400, esperado 400)
[OK] Crear fichero en directorio de otro usuario (HTTP 401, esperado 401)
[OK] Listar directorio con un fichero (HTTP 200, esperado 200)
[OK] Conseguir fichero propio (HTTP 200, esperado 200)
[OK] Conseguir fichero privado sin token (HTTP 401, esperado 401)
[OK] Conseguir fichero privado de otro usuario (HTTP 401, esperado 401)
[OK] Conseguir fichero inexistente (HTTP 404, esperado 404)
[OK] Cambiar privacidad de fichero de otro usuario (HTTP 401, esperado 401)
[OK] Cambiar privacidad con valor no válido (HTTP 400, esperado 400)
[OK] Cambiar privacidad de fichero inexistente (HTTP 404, esperado 404)
[OK] Hacer público el fichero (HTTP 200, esperado 200)
[OK] Conseguir fichero público sin token (HTTP 200, esperado 200)
[OK] Hacer privado el fichero (HTTP 200, esperado 200)
[OK] Conseguir fichero de nuevo privado sin token (HTTP 401, esperado 401)
[OK] Borrar fichero de otro usuario (HTTP 401, esperado 401)
[OK] Borrar fichero (HTTP 200, esperado 200)
[OK] Borrar fichero ya borrado (HTTP 404, esperado 404)
[OK] Listar directorio tras borrar (HTTP 200, esperado 200)
Resultado: 25/25 pruebas superadas
e5286609-11-9-11-~/Practical_lab/Entrega$
```

```
Terminal - e5286609-11-9-11-~/Practical_lab/Entrega
File Edit View Terminal Tabs Help
e5286609-11-9-11-~/Practical_lab/Entrega$ docker build -f Dockerfile -t entrega-user .
[+] Building 4/4
[+] Image entrega-user:api Built
[+] Container entrega-user:api-1 created
[+] Container entrega-file:api-1 created
Attaching to file-api-1, user-api-1
A new release of pip is available: 25.0.1 ->
To upd... Leer más
```

7. Limitaciones

Sobre las limitaciones del código, somos conscientes de que el token no caduca y depende de un secreto compartido que está dentro del proyecto, lo cual no es muy seguro, pero el propio enunciado dice que en un caso real se usaría JWT.